

へんすう 変数

JavaScriptの基礎：データの入れ物「変数」をマスターしよう

JavaScriptを学ぶ上で、最初の重要なステップが「変数」の理解です。

留学生の皆さんが日本語を学ぶとき、単語を覚えて整理するように、プログラミングでもデータを整理して名前をつける作業が必要です。変数を使いこなせるようになると、プログラムの中でデータを自由に再利用し、複雑な処理を組み立てることができるようになります。

では、まず、「変数とは何か」から、はじめましょう。

I. 変数とは何か：データを保存する「名前付きの箱」

プログラミングにおける「変数」とは、プログラムの中で扱う数値や文字列などのデータを保存しておくための「**名前のついた箱**」のようなものです。

空港で荷物を預けるシーンを想像してみてください。たくさんのスーツケースがあるとき、『どれが誰のものか？』『中身が何か？』を見分けるために「**名前のラベル**」を貼りますよね。これが変数です。ラベル（変数名）があるからこそ、後で必要なときに迷わずそのデータを取り出すことができます。

だいにゅう 1. 代入

「**名前のついた箱**」である「**変数**」の中にデータを**入れる**ことを、プログラミングでは「**代入**」と言います。

- 名前（ラベル）：変数名（例: `score`, `myName`）
- 箱の中身: データ（例: 数字の `100`, 文字の `"山田"`）

代入するときは、**=（イコール）** という記号を使います。ここで一番大切なルールがあります。

右側にあるデータを、左側の変数に入れる、

数学（すうがく）の「**=**」は「左と右が同じ」という意味ですが、プログラミングの「**=**」は「**入れる**」という動きを表します。

たとえば、テストの点数を `score` という変数に保存したいときは次のように書きます。

```
score = 100;
```

- これは、「**100**」という数字を、「**score**」という名前の箱に入れた という意味になります。

また、「再代入」とは、「一度、変数に値を代入したあとで、別の値を入れなおす」ことです。

一度、100という点数を入れたscoreを、あとで80点に変更する場合、下のように入ります。

```
score = 100; // 最初に100を代入（だいにゅう）します
// 途中（とちゅう）は省略（しょうりゃく）
score = 80; // 一度、100を入れた変数（へんすう）scoreに、80を再代入（さいだいにゅう）します
```

2. 変数宣言とキーワードの種類

変数を利用するには、まずシステムにその変数名を使うことを知らせる「変数宣言」が必要です。現代のJavaScriptでは、主に **const** と **let** という2つのキーワードを使用して宣言を行います。

なぜ変数を宣言するキーワードが2つあるのでしょうか？ 実は、JavaScriptには2種類の箱があります。プログラムを効率的に、そしてミスなく作るためには、これらを使い分けことが重要です。

キーワードは、プログラムが動いている途中で「値を変更するかどうか」で使い分けます。

letで宣言した変数は、あとで値を変えることができます。つまり「再代入が可能」です。反対に、**const**で宣言した変数は、あとで値を変えることができません。

種類	特徴	使いどき	例
let	中身を入れ替えられる箱	ゲームのスコアやクリックされた回数など、後で値が変わるもの	<code>const MAX_LEVEL = 99;</code>
const	一度入れたら変えられない箱	名前、誕生日、設定値など、ずっと同じもの	<code>let score = 0;</code> （あとで <code>score = 10;</code> と変えられる）

💡 プロの視点：なぜ **const** を優先するのか？

const で宣言された変数は、後から値を変更できません。一度決めた値が「変わらない」ことが約束されいてると、後からコードを読み返したときに迷わず、バグ（ミス）を減らすことができるからです。

ですから、みなさんは、できるだけ **const** を使いましょう。そして、後から値を変更しなければならないときだけ **let** を使いましょう。

※ むかしは **let** の代わりに **var** が使われていましたが、知らないうちに変数の中身が書き換えられてしまうかもしれないので、現在は、「使わないのがよい」とされています。

3. 変数の初期化

変数を宣言する際、同時に値を代入することを「初期化（しよきか）」と呼びます。

- 宣言と初期化を同時に行う例:

```
let myDog = "Rover"; // 宣言（せんげん）と同時（どうじ）に "Rover" という文字列（もじれつ）を入れる
const pi = 3.14;      // constは宣言（せんげん）するときに必（かなら）ず初期化（しよきか）が必要（ひつよう）
```

- 宣言だけ先に行う例（letのみ可能）:

```
let myName; // 箱（はこ）だけ用意（ようい）する。このときの中身（なかみ）は undefined（未定義（みていぎ））となる
myName = "Chris"; // あとから値（あたい）を代入（だいにゅう）する
```

4. 変数の名前の付け方

変数に名前を付けるときには、いくつかの守るべきルールと慣習があります。

- 大文字と小文字は区別される: Name と name は別の変数として扱われます。
- 使用できる文字: 半角英数字、アンダースコア（_）、ドル記号（\$）が使えますが、数字から始めることはできません。
- キャメルケース (camelCase): 複数の単語を繋げる場合、2つ目以降の単語の先頭を大文字にする書き方が一般的です。
 - 例: finalOutputValue、userName
- 意味のある名前: a や b といった一文字ではなく、count や isRaining など、中身が何であるか伝える英語の名前を付けるのがよい、とされています。

⚠ 留学生への重要アドバイス: 半角と全角

JavaScriptのコードを書く際、最も多いエラーの原因は全角文字の使用です。変数の名前にも、全角文字は使えません。すべて半角文字にしてください。

箱の使い方がわかったところで、次は、その箱に入れる「データの種類」について詳しく見ていきましょう。

以下のようなものがあります。

- 文字列型: let name = "山田"; (シングル ' ' または ダブル " " の引用符で囲む)
- 数値型: let age = 17; (引用符は付けない)
- 論理型: let iAmAlive = true; (true または false の真偽値)
- 配列: let myNameArray = ["Chris", "Bob", "Jim"]; (複数の値をまとめて管理するリスト)

- **オブジェクト**: `let dog = { name: "ポチ", breed: "ダルメシアン" };` (関連するデータと機能をひとつにまとめたもの)

ここでは「文字列型」「数値型」「論理型」について説明します。「配列」と「オブジェクト」については、あとの章で説明します。

II. 文字列型 (String)：テキストデータを扱う

ウェブページで「こんにちは」というメッセージを表示したり、ユーザーの名前を扱ったりする際に使われるのが「文字列」です。

文字列を作るには、テキストを引用符 (クォーテーション) で囲みます。

- シングルクォーテーション: `'こんにちは'`
- ダブルクォーテーション: `"こんにちは"`

どちらを使っても構いませんが、ひとつのプログラムの中で統一するのが良い習慣です。

文字の次は、計算に欠かせない「数値」の扱いに進みます。

III. 数値型 (Number) と特別な値 (NaN / Infinity)

JavaScriptでは、整数 (100) も小数 (3.14) も、すべて同じ「数値」型として扱います。

1. 算術演算子 (計算の記号)

数値を使って、算数と同じように計算ができます。

- `+` (足し算)
- `-` (引き算)
- `*` (掛け算)
- `/` (割り算)
- `%` (余り)

※ 算術演算子については、あとで、もっと詳しく説明します。

2. 特別な数値：NaNとInfinity

数値型には、注意が必要な特殊な値があります。

- **NaN (Not a Number)**：「数値ではない」状態です。例えば、文字列を無理やり計算しようとした時に発生します。
- **Infinity**：無限に大きな数値です。

計算の次は、プログラムが「正しいか間違いか」を判断するためのデータを見てみましょう。

IV. 論理型 (Boolean)：YesかNoかの判断基準

プログラムが「もし～なら、これをする」という制御（せいぎょ）を行う際、その土台となるのが「論理（ろんり）」型です。

1. 2つの値だけ

論理型には、true（真／正しい）と false（偽／間違い）の2つの値しか存在しません。

2. 比較と組み合わせ

比較演算子を使うと、結果として論理型の値が得られます。

- score === 100（スコアは100と厳密に等しいか？）
- age >= 20（年齢は20以上か？）

また、論理演算子を使って複数の条件を組み合わせることもできます。

演算子	意味	覚えるポイント
&&	～かつ～ (AND)	すべて true のときだけ true
	～または～ (OR)	どちらかが true であれば true
!	～ではない (NOT)	true と false を逆転させる

※ 比較演算子と論理演算子については、次の章で、もっと詳しく説明します

論理型を正しく理解すると、複雑なシステムの判断ロジックを整理して、ユーザーの入力に対して「正しく反応する」プログラムを書くことができるようになります。

V. 未定義型 (Undefined)：値が決まっていない状態

変数を作ったけれど、中身をまだ何も入れていないとき、JavaScriptはその状態を undefined と表現します。これは「未定義」型という特殊な型になります。「未定義」とは「まだ定義されていない」「まだ、どういうものが決まっていない、わからない」という意味です。

「空の箱」と「存在しない箱」の違い

初心者の方が間違いやすいのが、以下の違いです。

- undefined：箱（変数）はあるけれど、中身がまだ入っていない状態。
- ReferenceError：そもそも箱（変数）自体が存在していない、または名前を間違えている状態。

💡 デバッグのヒント 実行結果に undefined と表示されたら、「データの代入を忘れていないか?」「スペルミスをしていないか?」を疑ってみましょう。これはミスを特定するための重要な手がかりになります。

VI. 型の変換 (Type Conversion)

Webサイトの入力フォームから受け取るデータは、数字に見えても実際には「文字列」です。これを使って計算するには「型の変換」が必要です。

1. データの抽出: parseInt()

文字列の中から数字だけを取り出す便利な機能があります。

```
console.log(parseInt("55kg")); // 55 (数字の部分だけ取り出す)
console.log(parseInt("12.99ドル")); // 12 (整数部分だけ取り出す)
console.log(parseInt("A123")); // NaN (先頭が文字だと失敗する)
```

2. 安全な数値入力の受け取りフロー

プロの現場では、以下のように段階を踏んでデータを処理します。

1. 受け取る: prompt() など文字列として入力を受け取る。
2. 変換する: Number() や parseInt() で数値に変える。
3. チェックする: isNaN() で、正しく数字になったか確認する。

3. 🧐 学習者の味方: typeof 演算子

「今のこの変数は、何型なんだろう?」と迷ったときは、typeof を使いましょう。

```
const value = "123";
console.log(typeof value); // "string" と表示 (ひょうじ) される

const numValue = Number(value);
console.log(typeof numValue); // "number" と表示 (ひょうじ) される
```

自分の書いたコードが思ったとおりに動いているか、コンソールで typeof を使って確認するクセをつけましょう。

VII. まとめ: プログラミングの基礎を固める

変数と型の知識(ちしき)は、これからの学習の土台(どだい)になります。

💡 最重要ポイント

- 変数は「名前のついた箱」：データを記憶(きおく)し、何度でも使うために必要です。
- 基本は const を使う：安全のために const を使い、値を変えるときだけ let を使いましょう。
- 数値変換(すうちへんかん)を忘れずに：prompt() などで受け取ったデータは Number() や parseInt() で数字に変えてから計算しましょう。

⚠️ 初心者が間違えやすいポイント

- 引用符（クォーテーション）の忘れ：文字を書くときは必ず " " で囲んでください。囲まないと変数だと勘違(かんちが)いされてエラーになります。
- 全角(ぜんかく)文字の使用：プログラムのコード（let, const, ; など）は、必ず半角(はんかく)の英数字で書いてください。全角のスペースがひとつあるだけでも、プログラムは動かなくなります。

これで変数の基本は完璧(かんぺき)です。さあ、次に進みましょう！